CSE 554 Machine Learning Systems

Lecture 12 - Speculative Decoding

Baris Kasikci

BE BOUNDLESS

Class Overview

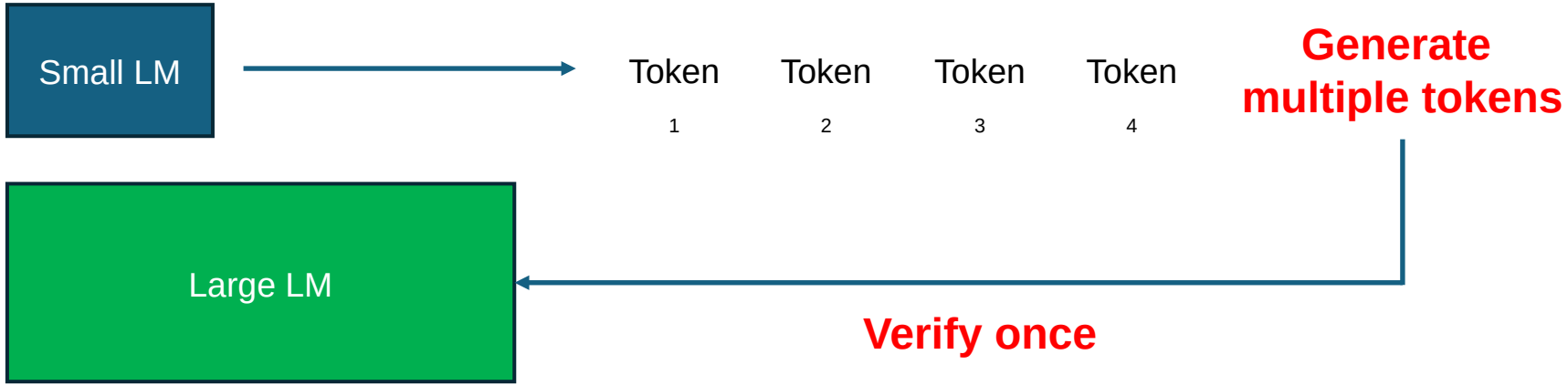
1. GPU Architecture and Programming
2. Transformers (Architecture and Implementation)
3. Performance Modeling
4. Quantization
5. Memory Management
6. NVIDIA Blackwell Architecture
7. Sparsity
8. Mixture of Experts (MoE)
9. **Speculative decoding**
10. Batching
11. Parallelism
12. End-to-end serving
13. Agentic systems

Key References

- Chen et al., Accelerating Large Language Model Decoding with Speculative Sampling, https://arxiv.org/abs/2302.01318
- Leviathan et al., Fast Inference from Transformers via Speculative Decoding, https://arxiv.org/abs/2211.17192

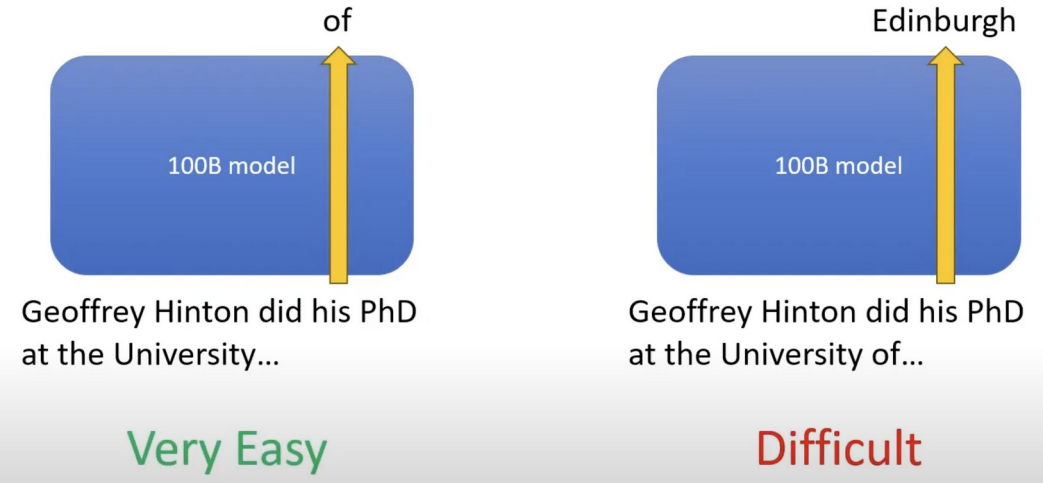
Speculative Decoding in a Nutshell

- Use a small LM (or a non auto-regressive generator) to generate tokens
  - Can be obtained via quantization, pruning, training from scratch, etc.
- Use a large LM to verify that the generated tokens are accurate
- Like speculative execution in CPUs, the small model may quickly generate many tokens that are mostly accurate



Key Observations Enabling Speculative Decoding

- LLM serving is **compute-bound** at large batch sizes
  - If the batch size is lower, LLM serving becomes **memory-bound**
  - A batch of quickly generated tokens can be verified in parallel at once
- A small (draft) LLM is quite accurate for most “easy” tokens
  - Most of the time, a large (target) LLM is not needed



https://www.youtube.com/watch?v=S-8yr_RibJ4

Draft Model Accuracy

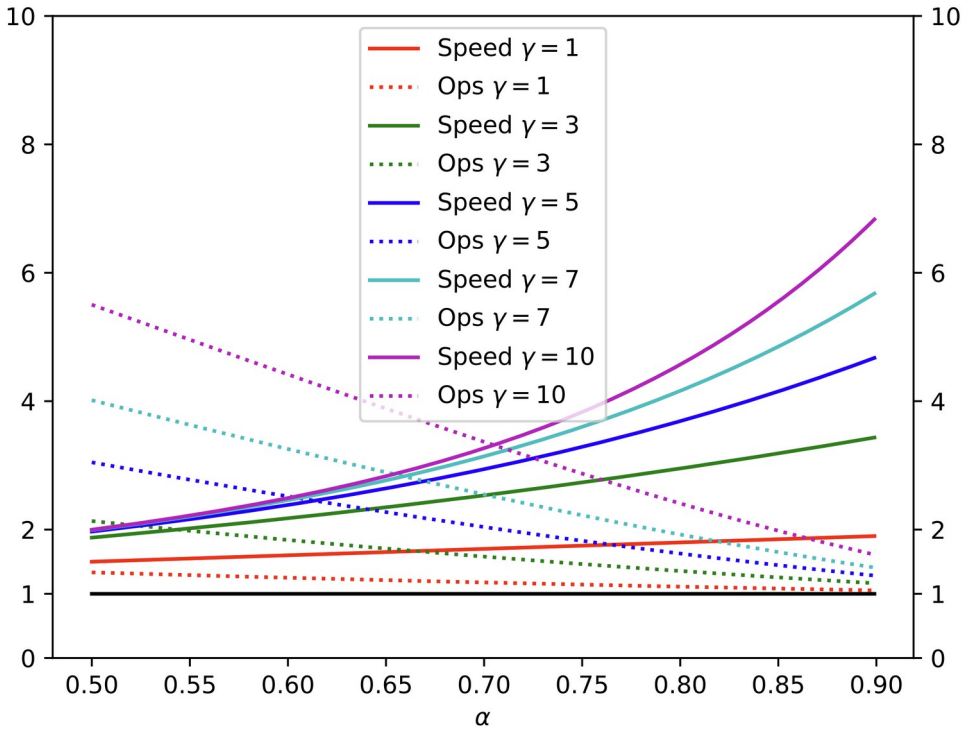
```
def below_threshold(l: list, t: int):
    """Return True if all numbers in the list l are below threshold t.
    >>> below_threshold([1, 2, 4, 10], 100)
    True
    >>> below_threshold([1, 20, 4, 10], 5)
    False
    """
    if isinstance(l, list):
        return True
    else:
        if t <= l < below_threshold(l, t):
            return True
    else:
        # If the first l element of l is an integer, then it is
        # the whole range of integers.
        if not isinstance(l[0], list):
            return True
    else:
        # If the first l element of l is a str, then it is
        # the whole string.
        if hasattr(l, 'findlen'):
            return findlen(l)
        return False


def thresh(t: int, max: int) -> int:
    """Return
```

Green (draft-1, N-gram)
Blue (draft-2)
Red (target)

https://arxiv.org/pdf/

Speedup from Speculative Decoding



α: a measure of how accurately the draft model represents the target model

γ: # of draft model predictions before verification

Leviathan et al. https://arxiv.org/pdf/2211.17192

Computing Next Token Probabilities in Parallel

```
# Project to vocabulary
# I: (seq_len, hidden_dim): (seq_len, 4096)
# O: (seq_len, vocab_size): (seq_len, 128256)
# For each token in the sequence, this gives a probability distribution over the vocabulary
logits = model_output.matmul(lm_head_weight.t())
# Pick the next token with the highest probability
sample_output = torch.argmax(logits, dim=1) # (seq_len, vocab_size): (seq_len, 128256)
# Return the next token following the last token in the input sequence
return sample_output[-1].item()
```

Next token probabilities for each token in the sequence (in one pass)

Parallel Verification

https://www.youtube.com/watch?v=S-8yr_RibJ4

Speculative Decoding Algorithm

- M_p: draft model
- M_q: target model

- pf = prefix, K = 5 tokens

$p_1(x) = M_p(pf)$ → x_1

$p_2(x) = M_p(pf, x_1)$ → x_2

…

$p_5(x) = M_p(pf, x_1, x_2, x_3, x_4)$ → x_5

The input to the model at each stage is the prefix and all the generated tokens

Two steps

1- Run the draft model K (e.g, 5) iterations

$p_1(x) = M_p(pf) \rightarrow x_1$

$p_2(x) = M_p(pf, x_1) \rightarrow x_2$

…

$p_5(x) = M_p(pf, x_1, x_2, x_3, x_4) \rightarrow x_5$

2- Run the target model once in parallel to verify them (feed the prefix)

$\boxed{q_1(x), q_2(x), q_3(x), q_4(x), q_5(x),}\ q_6(x)$

$= M_q(pf, x_1, x_2, x_3, x_4, x_5)$

All the tokens generated by the draft model

Target model only produces distributions, sampling is only done from the draft model

Generation and Verification

Tokens generated by the draft model

Draft model probabilities

Target model probabilities

Token	x1	x2	x3	x4	x5
	dogs	love	chasing	after	cars
p(x)	0.8	0.7	0.9	0.8	0.7
q(x)	0.9	0.8	0.8	0.3	0.8

Rejection Sampling

This is the process that'll decide if the tokens generated by the draft model will kept or rejected

Token	x1	x2	x3	x4	x5
	dogs	love	chasing	~~after~~	~~cars~~
p(x)	0.8	0.7	0.9	0.8	0.7
q(x)	0.9	0.8	0.8	0.3	0.8
	□	□	□ (0.8/0.9)	□	

Case 1: If q(x) ≥ p(x), accept

- Target model is even more confident than the draft model that it is correct

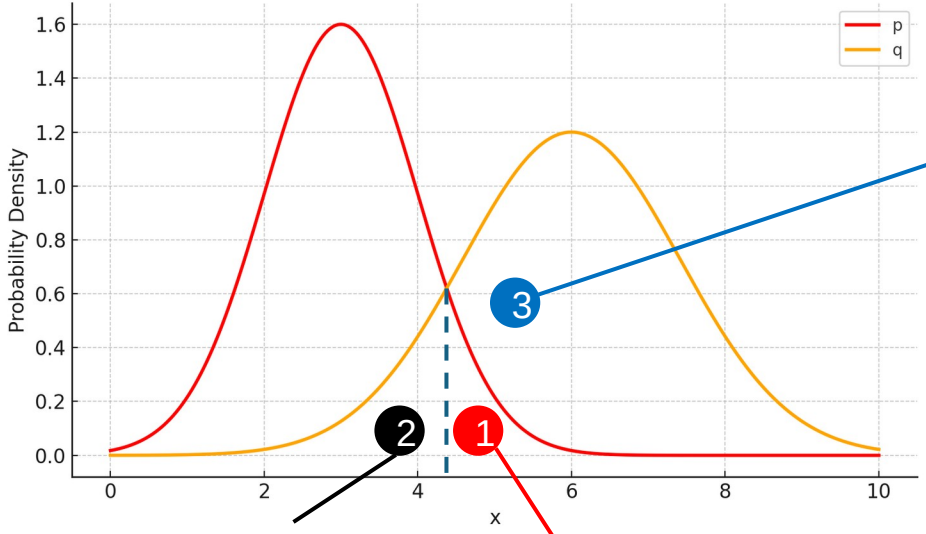
Case 2: If q(x) < p(x), then accept with probability $\frac{q(x)}{p(x)}$

What to do in case of rejection?

- We can sample from q(x), since we have the distribution
- Instead, we sample from (q(x) - p(x)) + means that we won't sample from here

Why does rejection sampling work?

- Goal: sample a token from q(x), while actually sampling from p(x)



q(x) < p(x), if a rejection decision is made, sample from $(q(x) - p(x))_+$

q(x) < p(x) accept with prob q(x)/p(x)

q(x) ≥ p(x) accept

What if all Tokens are Accepted?

- We sample from q(x): $q_6(x)$ in our example
  $q_1(x)$, $q_2(x)$, $q_3(x)$, $q_4(x)$, $q_5(x)$, $q_6(x)$

Token	x1	x2	x3	x4	x5
	dogs	love	chasing	after	cars
p(x)	0.8	0.7	0.9	0.8	0.7
q(x)	0.9	0.8	0.8	0.3	0.8

How many tokens are generated in one pass?

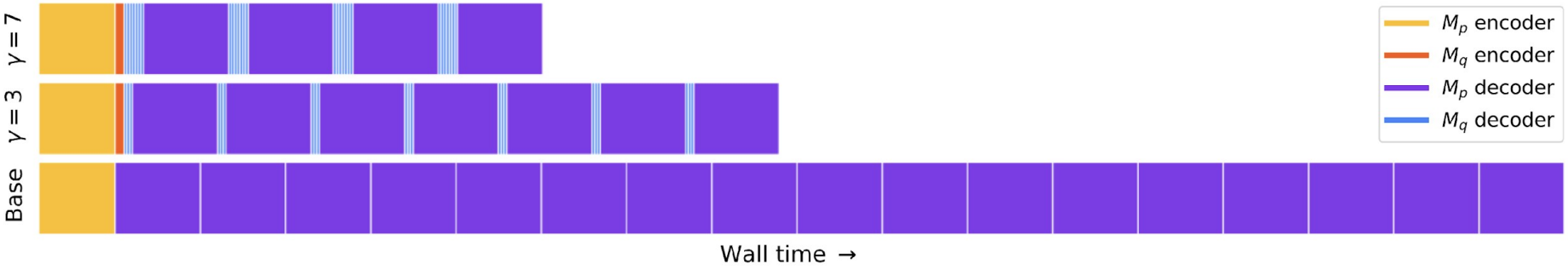
Token	x1	x2	x3	x4	x5
	dogs	love	chasing	after	cars
p(x)	0.8	0.7	0.9	0.8	0.7
q(x)	0.9	0.8	0.8	0.3	0.8

Worst case: first token is rejected -> 1 token generated

Best case: all tokens accepted -> K+1 tokens generation

The worst case does not slow down the algorithm as a forward pass generates one token

Benefits of speculative sampling



γ: # of draft model
predictions before verification

Diminishing returns from increased γ

Speculative decoding in action

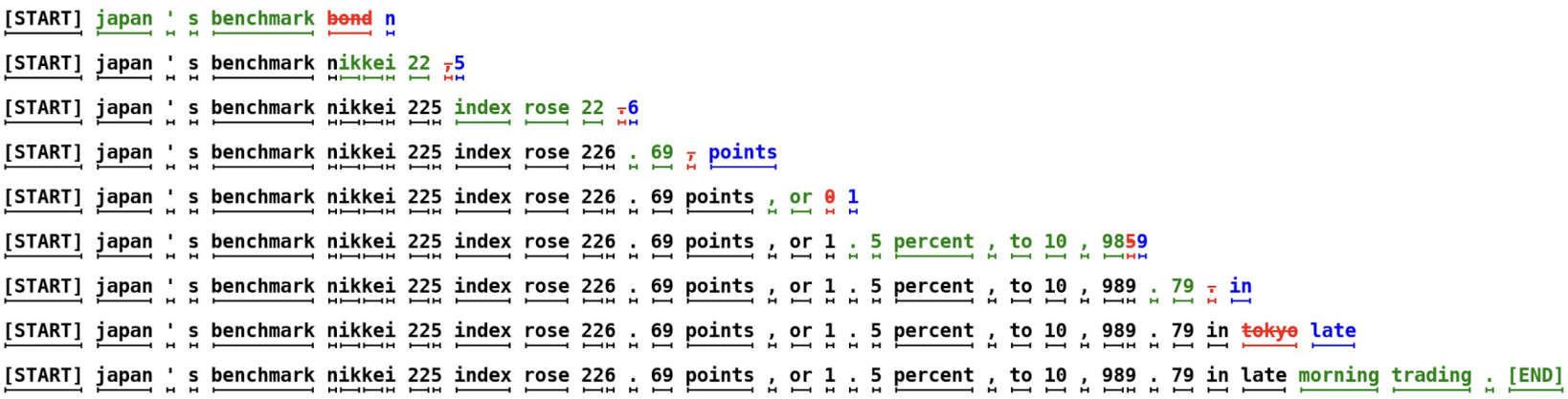
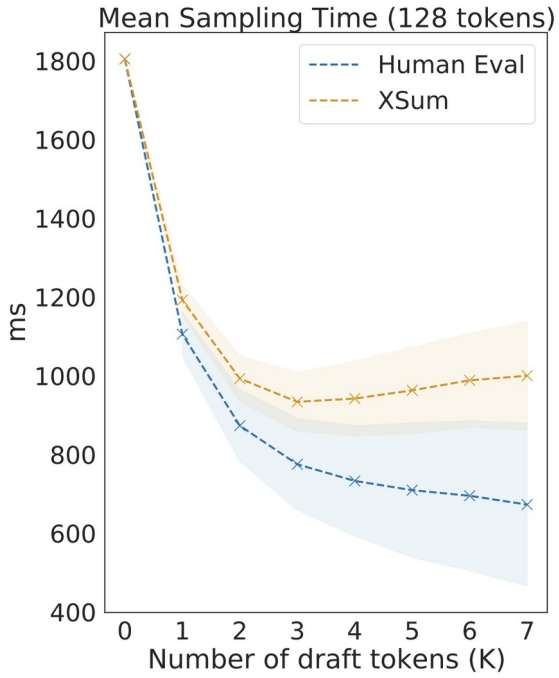


Figure 1. Our technique illustrated in the case of unconditional language modeling. Each line represents one iteration of the algorithm. The **green** tokens are the suggestions made by the approximation model (here, a GPT-like Transformer decoder with 6M parameters trained on lm1b with 8k tokens) that the target model (here, a GPT-like Transformer decoder with 97M parameters in the same setting) accepted, while the **red** and **blue** tokens are the rejected suggestions and their corrections, respectively. For example, in the first line the target model was run only once, and 5 tokens were generated.

Results

TASK	M_q	TEMP	γ	α	SPEED
ENDE	T5-SMALL ★	0	7	0.75	**3.4X**
ENDE	T5-BASE	0	7	0.8	2.8X
ENDE	T5-LARGE	0	7	0.82	1.7X
ENDE	T5-SMALL ★	1	7	0.62	**2.6X**
ENDE	T5-BASE	1	5	0.68	2.4X
ENDE	T5-LARGE	1	3	0.71	1.4X
CNNDM	T5-SMALL ★	0	5	0.65	**3.1X**
CNNDM	T5-BASE	0	5	0.73	3.0X
CNNDM	T5-LARGE	0	3	0.74	2.2X
CNNDM	T5-SMALL ★	1	5	0.53	**2.3X**
CNNDM	T5-BASE	1	3	0.55	2.2X
CNNDM	T5-LARGE	1	3	0.56	1.7X



Google Research

DeepMind

https://arxiv.org/pdf/2211.17192

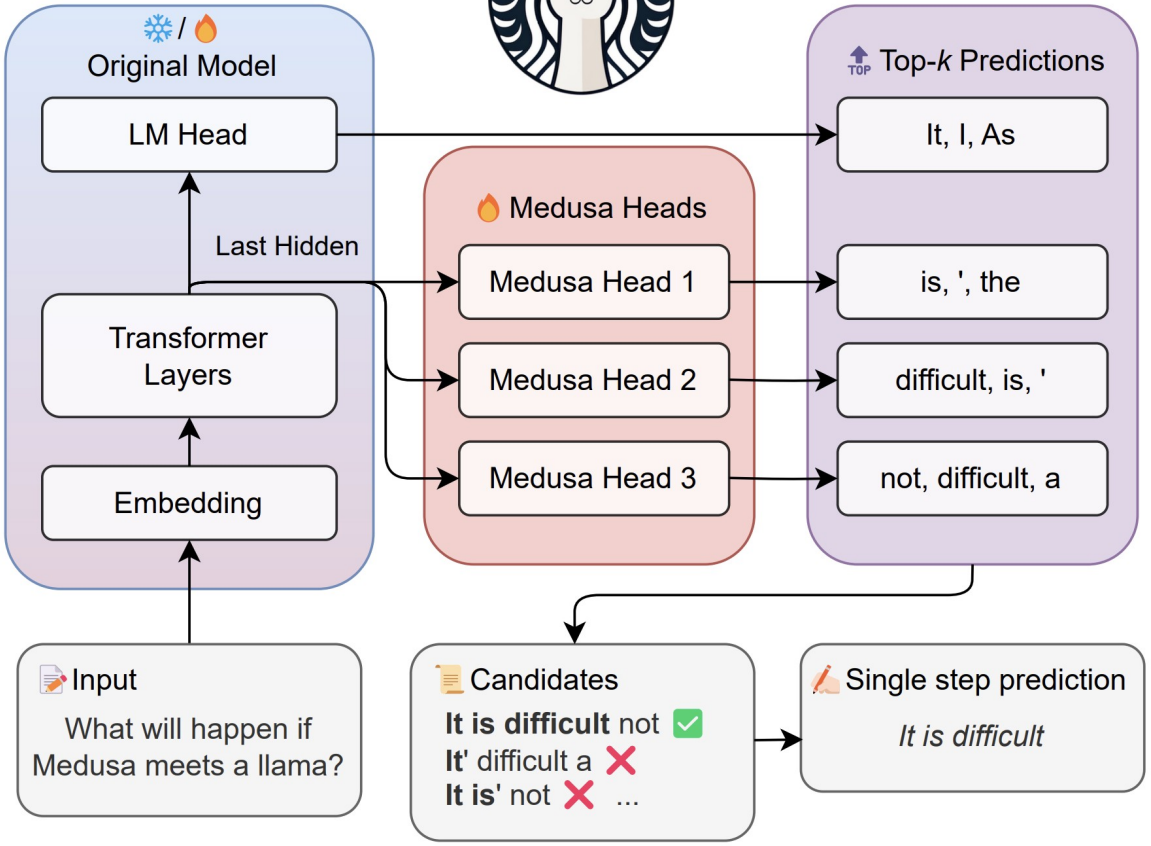
https://arxiv.org/pdf/2302.01318

Medusa

- Add a few additional heads to predict tokens
- Easy to train the new heads with a basic GPU
- Easy to serve
  - Same kind of parallelism (e.g., tensor parallelism) can be used
- Good speedup
  - ~3x

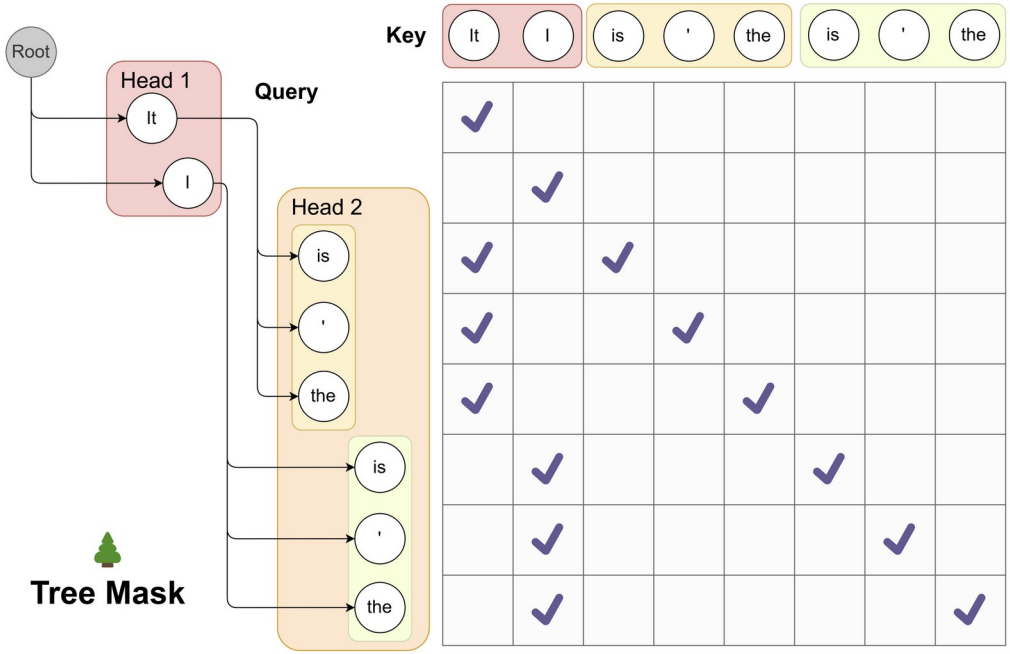
https://arxiv.org/abs/2401.10774

Design


Original Model
LM Head
Last Hidden
Transformer Layers
Embedding
Medusa Heads
Medusa Head 1
Medusa Head 2
Medusa Head 3
Top-k Predictions
It, I, As
is, ', the
difficult, is, '
not, difficult, a
Input
What will happen if Medusa meets a llama?
Candidates
It is difficult not
It' difficult a
It is' not ...
Single step prediction
It is difficult

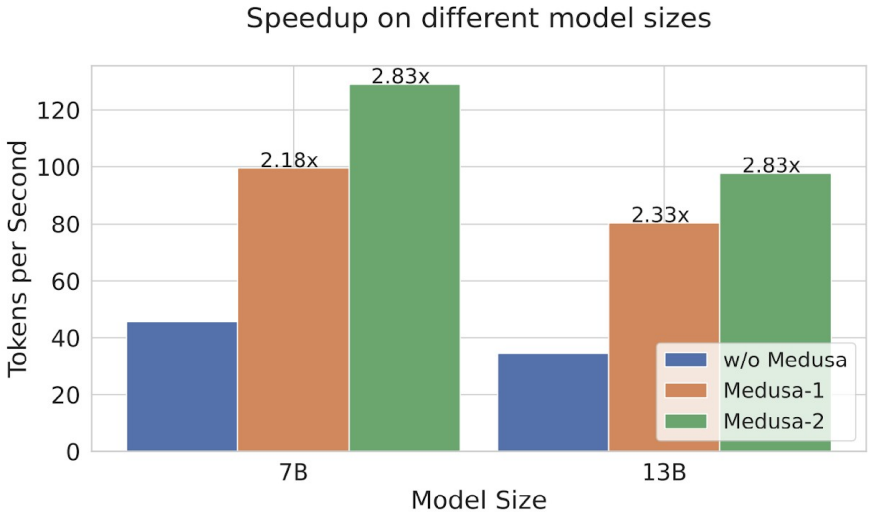
Tree Attention

- Heads provide different token candidates, forming different candidate sequences (It is, I is, etc.)
- Each sequence is a branch in the tree
- Tree attention (and the mask) allows each token to only attend to its predecessors
- Multiple sequences can be batched and verified in one forward pass

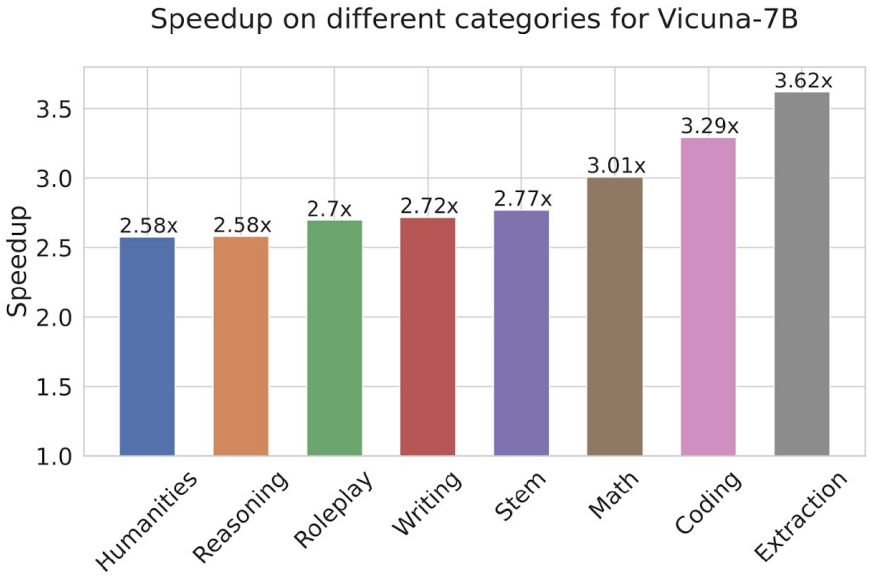


Evaluation

- Medusa-1: Medusa is directly fine-tuned on top of a frozen backbone LLM
- Medusa-2: Medusa is fined tuned together with the backbone LLM
  - Needs a special training recipe to preserve model's training capabilities



(a)



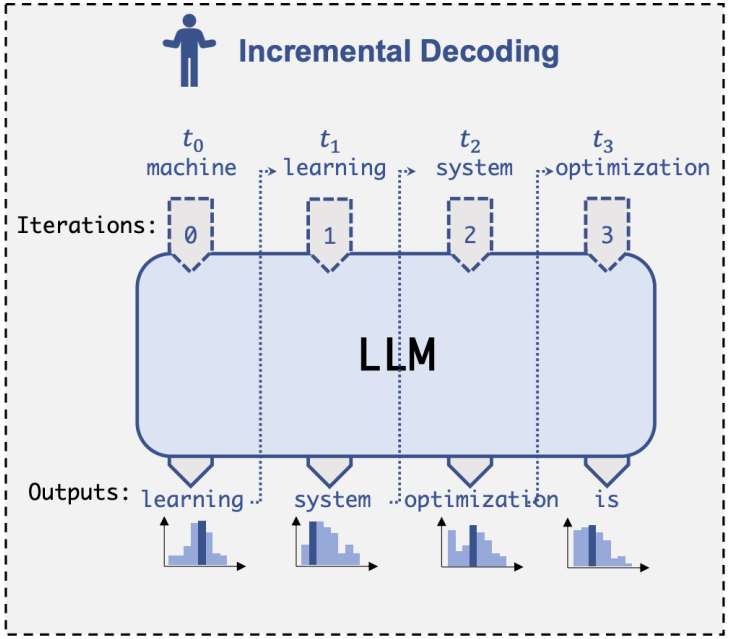
(b)

SpecInfer

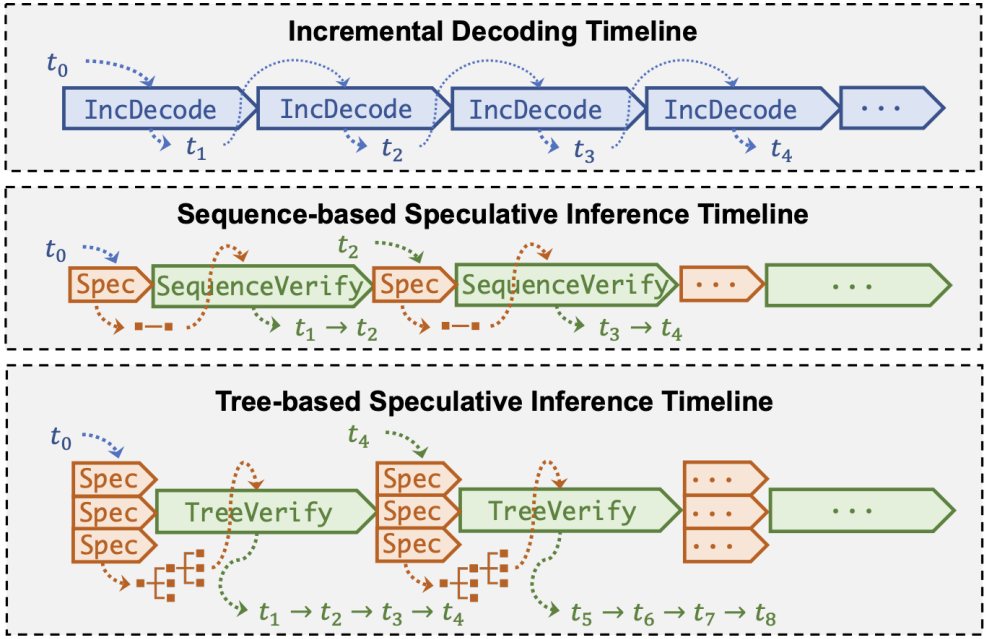
- One challenge with the draft model is training an accurate one
  - Draft model is much smaller, and a single draft model may not provide enough “coverage”
- Key idea: use multiple draft models simultaneously
  - Creates a tree of sequences
  - Can be verified simultaneously
    - Recall: we can batch verification sequences if we are in the memory-bound regime

Miao et al., SpecInfer: Accelerating Generative Large Language Model Serving with
Tree-based Speculative Inference and Verification

Incremental/Sequence-based/Tree-based Inference



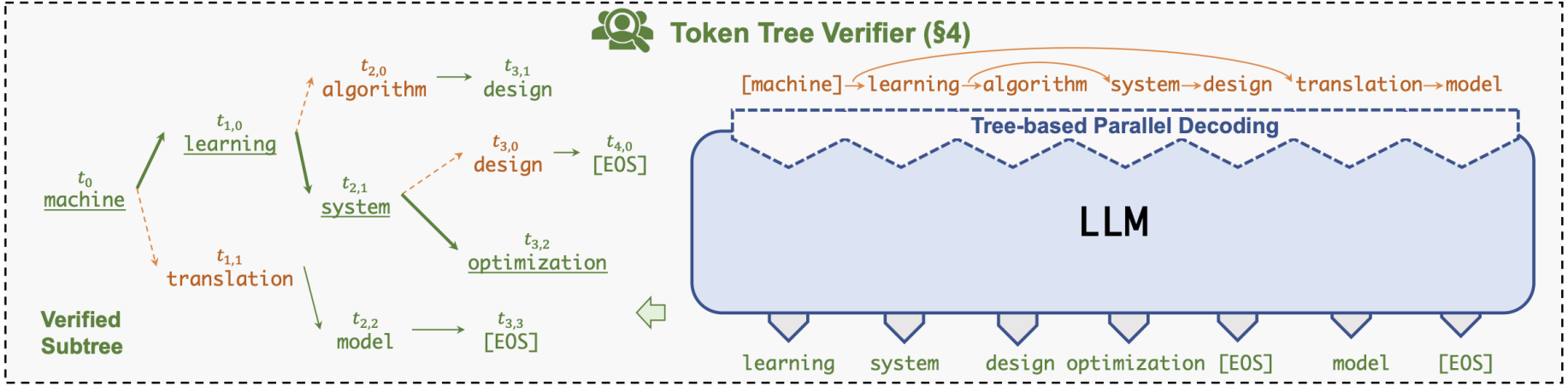
(a) Incremental decoding.



(b) Timeline Comparison.

Token tree verification

- Token tree verifier uses topology-aware causal mask to apply attention in a manner aware of the tree topology
  - This allows batching the verification requests



Evaluation

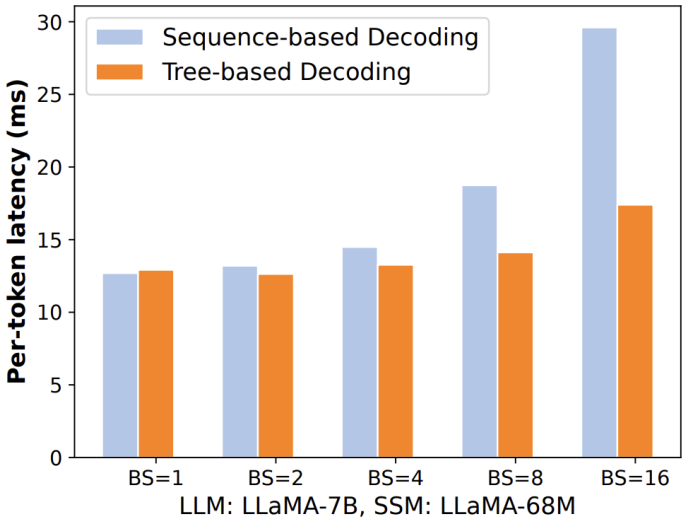


Figure 11. Comparing SpecInfer's tree-based parallel decoding with the sequence-based decoding mechanism employed by existing LLM inference systems.

Table 3. Average number of tokens verified by SpecInfer in a stochastic decoding step with different sampling algorithms. We use LLaMA-7B and LLaMA-68M as the LLM and SSM. Each token tree has a width of 5 and a depth of 8.

	Naive Sampling	**Multi-Step Spec. Sampling**	**Improvement**
Alpaca	1.87	2.38	1.27×
CP	1.80	2.28	1.26×
WebQA	1.73	2.21	1.28×
CIP	1.79	2.29	1.28×
PIQA	1.73	2.21	1.28×